

Задача 8: Оптимизация симуляции упругих столкновений

Цель задачи

Проявить знания в подходах к оптимизации программ на C++ и порядке их применения. Сравнить эффективность каждой оптимизации через количественные измерения времени инициализации и выполнения шагов симуляции.

Навыки студента

- Проводить систематический анализ производительности приложения
- Использование утилит для выявления проблем производительности
- Применять алгоритмические оптимизации
- Использовать низкоуровневые оптимизации
- Реализовывать zero-cору подходы при работе с файлами
- Проектировать измеримые архитектуры для сравнения производительности
- Анализировать компромиссы между временем инициализации и временем выполнения

Требования к выполнению задачи

В данной задаче требуется написать консольное приложение код которого демонстрирует различные подходы и этапы оптимизации программы.

Для оптимизации используется следующая задача:

- Разработать систему для пошаговой симуляции движения тел с упругими столкновениями.
- Основные требования:
 - Чтение начального состояния из файла
 - Пошаговая симуляция упругих столкновений между телами
 - Сохранение состояния при остановке в формате, совместимом с входным
 - Поддержка больших сцен (10,000+ тел) с высокой частотой столкновений
- Физическая модель:
 - Тела: сферы с массой, радиусом, позицией, скоростью
 - Столкновения: абсолютно упругие (сохранение энергии и импульса)
 - Гравитация: отсутствует
 - Пространство: 3D, ограниченное точностью вычислений выбранного типа презентации дробных чисел
 - Шаги симуляции производятся в реальном или ускоренном времени (т.е. один шаг симуляции должен рассчитывать как минимум движения в течении времени, который выполнялся предыдущий шаг симуляции, см. "DeltaTime problem")
- Требуется по запросу / таймауту демонстрировать корректность физической модели: Отслеживается сохранение импульса и энергии системы

Начальные условия (не оптимизированное решение):

- Чтение начального состояния из текстового файла и запись в текстовый файл
- Алгоритм попарной проверки коллизий тел

Для демонстрации примененных оптимизаций требуется оформлять код каждого этапа оптимизации в отдельный класс "движка симуляции". Требуется реализовать как минимум 4 движка (не включая базовый, не оптимизированный вариант), применяющих различные оптимизации.

В качестве метрик для сравнения оптимизаций используются (среднее / максимальное / минимальное / дисперсия):

- Время инициализации
- Время выполнения шага на прогреве
- Время выполнения шага в рабочем режиме

- Время записи конечного состояния в файл

Вывод метрик требуется организовать путем, не влияющим или минимально влияющим на основную задачу.

Формат файла состояния не фиксирован (кроме не оптимизированного случая) и может выбираться студентом самостоятельно, исходя из цели данной задачи.

Отдельно требуется продемонстрировать работу программы с режимами оптимизации компилятора:

- Без оптимизации
- С оптимизацией по размеру
- С максимальной оптимизацией

А также с режимом оптимизации при линковке (LTO).

Подготовительные материалы

Напоминание о классах и видах оптимизаций:

- Алгоритмические оптимизации (Uniform Grid, Octree/KD-tree, Sweep-and-Prune, Spatial hashing)
- Оптимизация подсчета столкновения (Векторизация вычислений (SIMD), Предвычисление часто используемых величин, Branchless programming для условных переходов)
- Оптимизации работы с памятью (Array-of-Structs против Struct-of-Arrays, Обработка данных блоками, уместающимися в кэш, Prefetching для предварительной загрузки данных)
- Файловый ввод/вывод (Zero-Copy, Copy-On-Write, Dirty Pages, Binary file data, memory mapped IO)
- Многопоточная обработка (Разделение задач между потоками, Task-based параллелизм для балансировки нагрузки, Lock-free структуры для разделяемых данных)

Выявление bottlenecks производительности:

- CPU cache utilization (cache hit / miss)
- CPU branch prediction
- Memory access patterns (page hits / misses)
- Memory utilization (pages vs huge pages)
- Memory reuse (zero-copy)
- IO speed (single thread R/W vs multi thread R/W)
- IO access patterns (sequential vs random)

Ссылки на материалы и справочные данные

Тестовые сценарии:

- Малое количество тел (100-500) - для отладки физики
- Среднее количество тел (1000-5000) - для тестирования оптимизаций
- Большое количество тел (10000+) - для оценки производительности
- Различные распределения (равномерное, кластерное, случайное)

Инструменты профилирования:

- Linux: perf, valgrind, gprof
- Windows: VTune, Windows Performance Analyzer
- Кросс-платформенные: Google Benchmark, Tracy profiler

Физическая модель:

- Упругие столкновения - математическая основа:
 - [Elastic Collision - Wikipedia](#)
 - [3D Elastic Collision Equations](#)
- Численные методы для симуляции:
 - [Verlet Integration](#)
 - [Time Step Adaptation Techniques](#)

Алгоритмические оптимизации:

- Пространственное разбиение:
 - [Spatial Hashing](#)
 - [Octree for Collision Detection](#)
- Оптимизации памяти:

- [Data-Oriented Design](#)
- [CPU Cache Optimization](#)

Технологии C++20:

- SIMD программирование:
 - [Intel Intrinsics Guide](#)
 - [C++ SIMD Libraries Comparison](#)
- Memory-mapped файлы:
 - [std::filesystem and mmap](#)
 - [Boost.Interprocess Memory Mapped Files](#)

Инструменты измерения производительности:

- Бенчмаркинг:
 - [Google Benchmark Library](#)
 - [C++ High Resolution Clock](#)
- Профилирование:
 - [Perf Linux Profiler](#)
 - [Tracy Profiler](#)

Референсные реализации:

- [NVIDIA Flex Physics SDK](#)
- [Bullet Physics Library](#)
- [Box2D Physics Engine](#)

Статьи и исследования:

- [Optimizing Collision Detection in Games](#)
- [Memory Access Pattern Optimization](#)